PGR111, solutions week 07

Task 1 - Concepts & Terminology

Briefly explain the following terminology and database concepts:

- ER Model
- ER model notifications: UML & Crow's Foot
- 1:1, 1:M and M:M relationships
- Composite entities

Note: Keep your answers. They might become helpful later in the semester? :-)

See textbook or google solutions for this.

Task 2 – ER Modelling

For modelling, you have a couple options:
a. Use old-fashioned pen and paper
b. Use an online tool, like Gliffy, Lucidchart, app.diagrams.net
Note: If you want to make a model for an existing DB, you can also use File -> New Model in DBeaver.

Imagine you have been hired to design the database solution for a new online shop.
Create an ER model for this.

They have the following thoughts regarding the structure of their online store:

1. They want to store customer data in their database, so they can send out advertisements, and so that customers don't need to re-enter all their personal information every time they want to place an order. Customers should have a unique id, as well as name and an address.

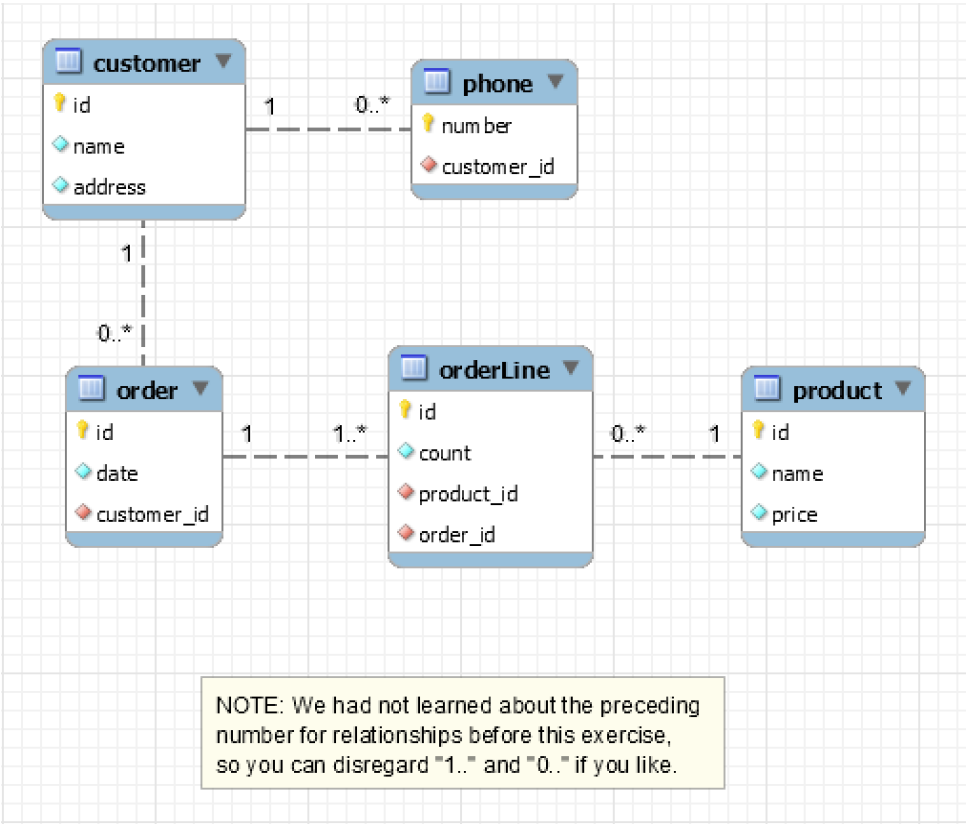
2. Customers should be able to store their telephone numbers so that they can be called in the event of delivery problems or the like. Each customer should be able to have several telephone numbers. (But they are also allowed to not register a phone at all.)

3. The products should be stored with id, name and price. For now, the store sells downloadable products only, so there is no reason to keep track of an inventory: As all goods are downloaded, everything product can be sold infinitely many times. (Smart, right? At least the store thinks so.)

4. Customers' orders must be registered in the database. Customers can register as many orders as they like. An order should contain a unique id and a date for when it was created.

5. *Difficult:* Each order can contain as many different items as the customer wants, and as many copies of each item as the customer wants. (Limited within reason – setting a maximum of several hundred or a few thousand is ok.) Each item can appear on many orders.

Part 1-5:



«Yellow key symbol» means PK in the above figure.

«Red diamond» means FK in the above figure.

Task 3 – Additional ER model tasks

The following tasks are borrowed from online sites. (with URL references listed). Draw ER diagrams for these cases.

a) From https://cs-uob.github.io/COMSM0085/exercises/part1/db1/er-diagram.html

> The University of Bristol Hoverboard Society (HovSoc) wants to create a database to manage its membership and events. Each member has a name, an optional student number, a contact e-mail address and a hoverboard riding skill level (represented as an integer, minimum 0). We assume that e-mail addresses are unique among members.

The committee consists of some of the members, each of which has a unique committee role. We assume that committee roles do not change during the year and that each committee role must be filled every year.

An event has a date, a name, a location, an optional description and an organiser who must be a society member (not necessarily a committee member). An event is attended by a set of members. There is never more than one event at the same location on the same date but event names are not unique.

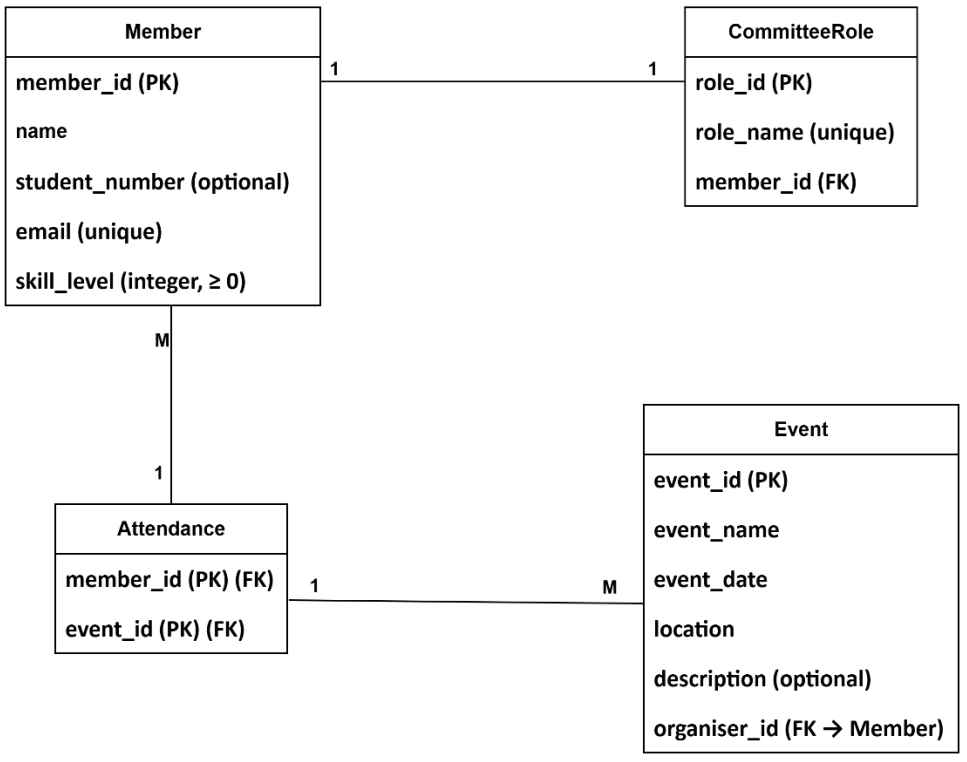
Don't have a solution for this one. We'll look at it together in the next lesson.

Example solution: Task 3 (a)

- Entities, attributes and relationships:
  - **Member (member_id** (PK), name, student_number (optional), email (unique), skill_level (integer, ≥ 0)). Each member has unique email and can be part of events or committee.
  - **CommitteeRole (role_id** (PK), role_name (unique), member_id (FK → Member.member_id)). Each role is filled by exactly one member, and each member may hold **at most one** role. (Relationship: 1–1 between Member and CommitteeRole.)
  - **Event (event_id** (PK), event_name, event_date, location, description (optional), organiser_id (FK → Member.member_id)). Organizer must be a Member, and there can be only one event per location+date combination (unique constraint).
  - **Attendance (composite entity- connecting Member and Event entities) (member_id** (FK → Member.member_id), **event_id** (FK → Event.event_id) → **PK** = (member_id, event_id)) Represents the many-to-many relationship between Members and Events.

- ER diagram:

Draw.io (custom list item for the Hoverboard Society database)



b) 2 extra ER diagram exercises (w/solutions below them, so be careful not to look ahead!) can be found here: https://opentextbc.ca/dbdesign01/back-matter/appendix-b-erd-exercises/

Solutions for these on their website (as noted in the exercise text above).

Task 4 – Do remaining, previous SQL tasks

Even though we don't focus on SQL right now, it will be a large part of the exam. If you have any tasks left from the earlier SQL exercises, do them now! :-)

See solutions for earlier weeks.